

PSC MID-II NOTES (R22)

UNIT - III

Preprocessor Directives: File Inclusion, Macros

Q1. Define a preprocessor macro swap (x, y) that will swap two arguments x and y of given type

A.

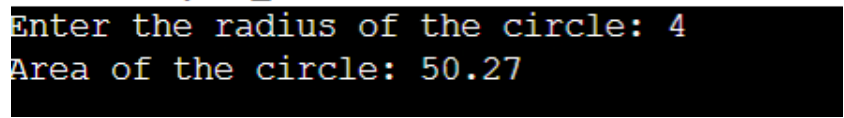
```
#include <stdio.h>
// Macro to swap two values of a given type
#define swap(type, x, y) { type temp = x; x = y; y = temp; }
int main() {
    int a = 5, b = 10;
    printf("Before swapping: a = %d, b = %d\n", a, b);
    // Using the swap macro
    swap(int, a, b);
    printf("After swapping: a = %d, b = %d\n", a, b);
    return 0;
}
```

```
Before swapping: a = 5, b = 10
After swapping: a = 10, b = 5
```

Q2. Write a C Program to find area of a circle, passing arguments to macros.

A.

```
#include <stdio.h>
#define PI 3.14159
#define area_of_circle(radius) (PI * (radius) * (radius))
int main() {
    float radius;
    // Input radius from the user
    printf("Enter the radius of the circle: ");
    scanf("%f", &radius);
    // Calculate and print the area using the macro
    printf("Area of the circle: %.2f\n", area_of_circle(radius));
    return 0;
}
```



Enter the radius of the circle: 4
Area of the circle: 50.27

Q3. List out Conditional directives of preprocessor with its syntax.

A.

Conditional directives in the preprocessor include:

 HTML Content

#if, #else, #elif, #endif:

c

[Copy code](#)

```
#if condition // Code to include if condition is true #elif
another_condition // Code to include if another_condition is true
#else // Code to include if none of the conditions is true #endif
```

#ifdef and #ifndef:

c

[Copy code](#)

```
#ifdef MACRO_NAME // Code to include if MACRO_NAME is defined
#endif #ifndef MACRO_NAME // Code to include if MACRO_NAME is not
defined #endif
```

#define:

c

[Copy code](#)

```
#define MACRO_NAME
```

#undef:

c

[Copy code](#)

```
#undef MACRO_NAME
```

#error:

c

[Copy code](#)

```
#if !defined(MACRO_NAME) #error "Macro MACRO_NAME is not defined."
#endif
```

Q4. C code to demonstrate Celsius to Fahrenheit using macros

A.

```
#include <stdio.h>
#define CELSIUS_TO_FAHRENHEIT(celsius) ((celsius) * 9/5 + 32)
int main() {
    float celsius;
    // Input temperature in Celsius from the user
    printf("Enter temperature in Celsius: ");
    scanf("%f", &celsius);
    // Convert and print the temperature in Fahrenheit using the macro
    printf("Equivalent temperature in Fahrenheit: %.2f\n", CELSIUS_TO_FAHRENHEIT(celsius));
    return 0;
}
```

```
Enter temperature in Celsius: 100
Equivalent temperature in Fahrenheit: 212.00
```

Q5. What is a pre processor in c?

A. In C programming, the preprocessor is a phase of translation that occurs before the actual compilation of the source code. The preprocessor performs text manipulations on the source code based on preprocessor directives. It is a separate program that runs before the compiler processes the code.

The preprocessor is responsible for:

1. File Inclusion:

- The `#include` directive is used to include files into the source code. This allows code reuse by incorporating the contents of header files or other source files.

```
#include <stdio.h>
```

2. Macro Expansion:

- Macros are defined using `#define` and are expanded throughout the code. They provide a way to create simple code replacements.

```
#define PI 3.14159
```

3. Conditional Compilation:

- Conditional compilation directives (`#if`, `#ifdef`, `#ifndef`, `#else`, `#elif`, `#endif`) allow parts of the code to be included or excluded based on conditions.

```
#ifdef DEBUG  
// Debugging code  
#endif
```

4. Symbolic Constants:

- `#define` is used to define symbolic constants, making the code more readable and allowing for easy changes.

```
#define MAX_SIZE 100
```

5. Pragma Directive:

- The `#pragma` directive provides implementation-specific instructions to the compiler.

```
#pragma warning(disable: 1234)
```

The preprocessor directives start with a `#` symbol, and they are processed before the actual compilation of the code. The output after preprocessing is then passed to the compiler for further translation and generation of machine code.

Arrays: Defining an array, processing an array, passing arrays to functions.

Q6. What is an array? Write the salient features of an array and their uses and even passing to the function

A.

Array: An array is a collection of elements of the same data type stored in contiguous memory locations. Each element in an array is accessed by its index or subscript. Arrays provide a convenient way to store and manipulate a group of related data.

Salient Features of an Array:

1.Homogeneous Elements: All elements of an array must be of the same data type.

2.Fixed Size: Arrays have a fixed size that is defined at the time of declaration.

3.Contiguous Memory Allocation: Array elements are stored in adjacent memory locations.

4.Indexing: Elements in an array are accessed using an index or subscript, starting from 0.

5.Random Access: Array elements can be directly accessed using their index, allowing for fast retrieval.

Uses of Arrays:

1.Storing Multiple Values: Arrays are used to store multiple values of the same type in a single variable.

2.Efficient Data Access: Arrays provide direct and efficient access to individual elements using their indices.

3.Iteration: Arrays are often used for iteration, making it easy to traverse and manipulate a sequence of data.

4.Matrices and Multidimensional Arrays: Arrays can represent matrices and multidimensional structures.

Initialising an array at declaration and passing it to function

```
#include <stdio.h>
// Function to calculate and print the sum of an array
void printSum(int arr[], int size) {
    int sum = 0;
    // Calculate the sum of the array elements
```

```

for (int i = 0; i < size; i++) {
    sum += arr[i];
}
// Print the sum
printf("Sum of the array elements: %d\n", sum);
}

int main() {
    int numbers[] = {2, 4, 6, 8, 10}; //Initializing at the Time of Declaration
    // Calculate and print the sum of the array elements by calling the function
    printSum(numbers, 5);
    return 0;
}

```

Q7. How multidimensional array is different from one dimensional?

A.

One-Dimensional Array:

- A one-dimensional array is a collection of elements stored in a linear sequence.
- It has a single subscript/index to access elements.
- Declaration: `int arr[5];`

Multidimensional Array:

- A multidimensional array is an array of arrays, providing multiple indices to access elements.
- Common types include 2D arrays (matrices), 3D arrays, and so on.
- Declaration: `int matrix[3][3];`

Differences:

1.Access Mechanism:

- In a one-dimensional array, elements are accessed using a single index.
- In a multidimensional array, elements are accessed using multiple indices.

2.Declaration Syntax:

- One-dimensional array: `int arr[5];`
- Multidimensional array: `int matrix[3][3];`

3.Memory Representation:

- One-dimensional array elements are stored in a linear sequence.
- Multidimensional array elements are stored in a tabular or matrix-like structure with multiple rows and columns.

4.Application:

- One-dimensional arrays are suitable for linear data structures.
- Multidimensional arrays are used for representing matrices, tables, and other tabular structures.

Q8. Write a program to find the sum and average of 'n' given number using arrays.

A.

```
#include <stdio.h>
int main() {
    int n;
    // Input the number of elements
    printf("Enter the number of elements: ");
```

```
scanf("%d", &n);
int numbers[n], sum = 0;
// Input the numbers
printf("Enter %d numbers:\n", n);
for (int i = 0; i < n; i++) {
    printf("Enter number %d: ", i + 1);
    scanf("%d", &numbers[i]);
    // Calculate the sum
    sum += numbers[i];
}
// Calculate and print the sum and average
printf("Sum of the numbers: %d\n", sum);
printf("Average of the numbers: %.2f\n", (float)sum / n);
return 0;
}
```

```
Enter the number of elements: 4
Enter 4 numbers:
Enter number 1: 6
Enter number 2: 5
Enter number 3: 4
Enter number 4: 2
Sum of the numbers: 17
Average of the numbers: 4.25
```

Multidimensional Arrays:

Example programs on matrix operations

Refer Unit-IV first 4 matrices questions

UNIT - IV

**Pointers: Pointer
Declarations, Passing
pointers to functions, NULL
pointer, Pointers and one-
dimensional Arrays, Dynamic
memory allocation,
operations on pointers,
pointers and
multidimensional arrays,
arrays of pointers.**

1. Write a C program to find the sum of two matrices. (In all the matrices question only a small piece of logic code differs for the respective question to perform. So instead of panicking try to compare the logics and understand)

A.

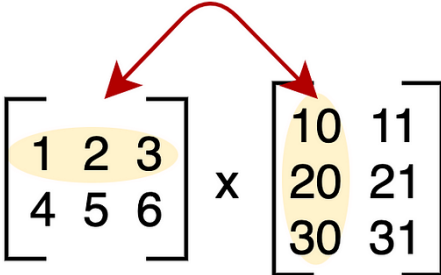
```
#include <stdio.h>
void addMatrices(int first[10][10], int second[10][10], int result[10][10], int rows, int
columns) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < columns; j++) {
            result[i][j] = first[i][j] + second[i][j];
        }
    }
}
int main() {
    int rows, columns;
    // Input matrix dimensions
    printf("Enter the number of rows: ");
    scanf("%d", &rows);
    printf("Enter the number of columns: ");
    scanf("%d", &columns);
    int firstMatrix[10][10], secondMatrix[10][10], sumMatrix[10][10];
    // Input elements of the first matrix
    printf("Enter elements of the first matrix:\n");
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < columns; j++) {
            scanf("%d", &firstMatrix[i][j]);
        }
    }
    // Input elements of the second matrix
    printf("Enter elements of the second matrix:\n");
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < columns; j++) {
            scanf("%d", &secondMatrix[i][j]);
        }
    }
    // Call the function to add matrices
    addMatrices(firstMatrix, secondMatrix, sumMatrix, rows, columns);
    // Display the result
    printf("Sum of the matrices:\n");
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < columns; j++) {
```

```
printf("%d\t", sumMatrix[i][j]);  
}  
printf("\n");  
}  
return 0;  
}
```

```
Enter the number of rows: 3  
Enter the number of columns: 3  
Enter elements of the first matrix:  
1      2      3  
4      5      6  
7      8      9  
Enter elements of the second matrix:  
1      2      3  
4      5      6  
7      8      9  
Sum of the matrices:  
2      4      6  
8      10     12  
14     16     18
```

2. Write a C program to find the multiplication of two matrices using functions?

A.



$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \times \begin{bmatrix} 10 & 11 \\ 20 & 21 \\ 30 & 31 \end{bmatrix}$$

$$= \begin{bmatrix} 1 \times 10 + 2 \times 20 + 3 \times 30 & 1 \times 11 + 2 \times 21 + 3 \times 31 \\ 4 \times 10 + 5 \times 20 + 6 \times 30 & 4 \times 11 + 5 \times 21 + 6 \times 31 \end{bmatrix}$$

$$= \begin{bmatrix} 10+40+90 & 11+42+93 \\ 40+100+180 & 44+105+186 \end{bmatrix} = \begin{bmatrix} 140 & 146 \\ 320 & 335 \end{bmatrix}$$

(Above image is just for idea how matrix multiplication is done)

```

#include <stdio.h>
void multiplyMatrices(int first[10][10], int second[10][10], int result[10][10], int rows1,
int columns1, int rows2, int columns2) {
for (int i = 0; i < rows1; i++) {
for (int j = 0; j < columns2; j++) {
result[i][j] = 0;
for (int k = 0; k < columns1; k++) {
result[i][j] += first[i][k] * second[k][j];
}
}
}
}
int main() {
int rows1, columns1, rows2, columns2;
// Input dimensions of the first matrix
printf("Enter the number of rows for the first matrix: ");
scanf("%d", &rows1);
printf("Enter the number of columns for the first matrix: ");
scanf("%d", &columns1);
// Input dimensions of the second matrix
printf("Enter the number of rows for the second matrix: ");
scanf("%d", &rows2);
printf("Enter the number of columns for the second matrix: ");
scanf("%d", &columns2);
if (columns1 != rows2) {

```

```
printf("Multiplication not possible. Number of columns in the first matrix should be equal  
to the number of rows in the second matrix.\n");  
return 0;  
}  
int firstMatrix[10][10], secondMatrix[10][10], productMatrix[10][10];  
// Input elements of the first matrix  
printf("Enter elements of the first matrix:\n");  
for (int i = 0; i < rows1; i++) {  
    for (int j = 0; j < columns1; j++) {  
        scanf("%d", &firstMatrix[i][j]);  
    }  
}  
// Input elements of the second matrix  
printf("Enter elements of the second matrix:\n");  
for (int i = 0; i < rows2; i++) {  
    for (int j = 0; j < columns2; j++) {  
        scanf("%d", &secondMatrix[i][j]);  
    }  
}  
// Call the function to multiply matrices  
multiplyMatrices(firstMatrix, secondMatrix, productMatrix, rows1, columns1, rows2,  
columns2);  
// Display the result  
printf("Product of the matrices:\n");  
for (int i = 0; i < rows1; i++) {  
    for (int j = 0; j < columns2; j++) {  
        printf("%d\t", productMatrix[i][j]);  
    }  
    printf("\n");  
}  
return 0;  
}
```

```

Enter the number of rows for the first matrix: 3
Enter the number of columns for the first matrix: 3
Enter the number of rows for the second matrix: 3
Enter the number of columns for the second matrix: 3
Enter elements of the first matrix:
1      2      3
4      5      6
7      8      9
Enter elements of the second matrix:
1      2      3
4      5      6
7      8      9
Product of the matrices:
30     36     42
66     81     96
102    126    150

```

3. Write a C program to find the transpose of matrices using functions?

A.

Transpose of a Matrix- examples

A	A ^T
$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$	$\begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix}$
$[5]$	$[5]$
$\begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \\ 7 & 8 \end{bmatrix}$	$\begin{bmatrix} 1 & 3 & 5 & 7 \\ 2 & 4 & 6 & 8 \end{bmatrix}$
$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$	$\begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & 9 \end{bmatrix}$

```

#include <stdio.h>
void transposeMatrix(int matrix[10][10], int rows, int columns, int result[10][10]) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < columns; j++) {
            result[j][i] = matrix[i][j];
        }
    }
}

```

```

}
int main() {
int rows, columns;
// Input matrix dimensions
printf("Enter the number of rows: ");
scanf("%d", &rows);
printf("Enter the number of columns: ");
scanf("%d", &columns);
int originalMatrix[10][10], transposedMatrix[10][10];
// Input elements of the matrix
printf("Enter elements of the matrix:\n");
for (int i = 0; i < rows; i++) {
for (int j = 0; j < columns; j++) {
scanf("%d", &originalMatrix[i][j]);
}
}
// Call the function to find the transpose
transposeMatrix(originalMatrix, rows, columns, transposedMatrix);
// Display the result
printf("Transposed matrix:\n");
for (int i = 0; i < columns; i++) {
for (int j = 0; j < rows; j++) {
printf("%d\t", transposedMatrix[i][j]);
}
printf("\n");
}
return 0;
}

```

```

Enter the number of rows: 3
Enter the number of columns: 3
Enter elements of the matrix:
1      2      3
4      5      6
7      8      9
Transposed matrix:
1      4      7
2      5      8
3      6      9

```

4. A square matrix is an identity matrix if all the principal or main diagonal elements are 1's and all the remaining

elements are O's. Write a C program that determines whether a square matrix is identity matrix or not.

A.

$$I_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$I_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$I_4 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$I_5 = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

```
#include <stdio.h>
int isIdentityMatrix(int matrix[10][10], int n) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if ((i == j && matrix[i][j] != 1) || (i != j && matrix[i][j] != 0)) {
                return 0; // Not an identity matrix
            }
        }
    }
    return 1; // Identity matrix
}

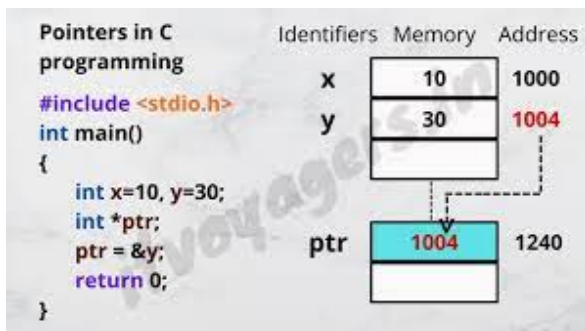
int main() {
    int n;
    // Input matrix dimension
    printf("Enter the dimension of the square matrix: ");
    scanf("%d", &n);
    int matrix[10][10];
    // Input elements of the matrix
    printf("Enter elements of the matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &matrix[i][j]);
        }
    }
}
```

```
// Check if the matrix is an identity matrix
if (isIdentityMatrix(matrix, n)) {
    printf("The matrix is an identity matrix.\n");
} else {
    printf("The matrix is not an identity matrix.\n");
}
return 0;
}
```

```
Enter the dimension of the square matrix: 3
Enter elements of the matrix:
1      0      0
0      1      0
0      0      1
The matrix is an identity matrix.
```

Q5. What is a pointer? Explain the reasons for using pointers?

A. A pointer is a variable that holds the memory address of another variable. It allows direct manipulation of memory, providing a way to access and manage data indirectly. Pointers in C provide flexibility and efficiency in memory management.



Reasons for using pointers and passing them to functions:
(Any 4-5 points)

1. **Dynamic Memory Allocation:** Pointers enable dynamic memory allocation using functions like malloc and free. This allows programs to allocate memory at runtime, adapting to changing requirements.
2. **Efficient Parameter Passing:** Passing pointers to functions is more efficient than passing the actual data, especially for large data structures. It avoids unnecessary copying, reducing the overhead associated with function calls.
3. **Direct Access to Memory:** Pointers allow direct access to memory locations, enabling efficient manipulation of data structures like arrays, linked lists, and dynamic data structures.
4. **Data Sharing between Functions:** Pointers facilitate sharing and modification of data across different functions. Changes made to the data through pointers in one function are reflected in the calling function.
5. **Efficient String Manipulation:** Pointers are commonly used for string manipulation, providing a flexible and memory-efficient way to work with character arrays.
6. **Array Operations:** Pointers simplify array operations by providing direct access to array elements. They are often used to iterate through arrays and manipulate their elements.

7. **Efficient Data Structures:** Pointers are essential for implementing complex data structures like linked lists, trees, and graphs, where dynamic memory allocation and direct memory access are crucial.

Q6. Explain various DMA functions with an example.

A.

Dynamic Memory Allocation (DMA):

Dynamic Memory Allocation refers to the process of allocating memory at runtime, during the execution of a program. In C programming, DMA is achieved using functions like malloc, calloc, realloc, and free. Unlike static memory allocation (where memory is allocated at compile time), dynamic memory allocation allows the program to adapt to varying memory requirements during runtime.

Memory Allocation Functions:

1. malloc (Memory Allocation):

- void *malloc(size_t size);
- Allocates a specified number of bytes of memory.
- Returns a pointer to the beginning of the allocated memory.
- The memory is not initialized.

```
int *arr = (int *)malloc(5 * sizeof(int));
```

2. calloc (Contiguous Allocation):

- `void *calloc(size_t num, size_t size);`
- Allocates memory for an array of elements (specified by `num`), each of size `size`.
- Initializes the allocated memory to zero.
- Returns a pointer to the beginning of the allocated memory.

```
int *arr = (int *)calloc(5, sizeof(int));
```

3.realloc (Re-allocation):

- `void *realloc(void *ptr, size_t size);`
- Changes the size of the previously allocated memory block.
- Returns a pointer to the new memory block.
- Used for resizing the memory block, and it may move the existing data to a new location.

```
int *arr = (int *)malloc(5 * sizeof(int));  
arr = (int *)realloc(arr, 10 * sizeof(int));
```

4.free (Deallocation):

- `void free(void *ptr);`
- Deallocates the memory block pointed to by `ptr`.
- Should be used to release memory allocated by `malloc`, `calloc`, or `realloc`.

```
int *arr = (int *)malloc(5 * sizeof(int));  
free(arr);
```

Memory Allocation Process:

1. Allocation:

- Use **malloc** or **calloc** to allocate memory.
- The allocated memory is usually a contiguous block.

2. Initialization (for calloc):

- If **calloc** is used, the allocated memory is initialized to zero.

3. Usage:

- Store and retrieve data in the allocated memory using the returned pointer.

4. Reallocation (if needed):

- Use **realloc** if the size of the allocated memory needs to be changed.

5. Deallocation:

- Use **free** to release the allocated memory when it is no longer needed.

Example:

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int *arr;
    // Allocate memory for an array of 5 integers
    arr = (int *)malloc(5 * sizeof(int));
    if (arr == NULL) {
        printf("Memory allocation failed.\n");
        return 1;
    }
    // Input values into the array
```

```
printf("Enter value %d: ", i + 1);
scanf("%d", &arr[i]);
}
// Display the values stored in the array
printf("Values stored in the array:\n");
for (int i = 0; i < 5; i++) {
printf("%d ", arr[i]);
}
// Deallocate the allocated memory
free(arr);
return 0;
}
```

In this example, memory is dynamically allocated for an array of integers, values are input, displayed, and then the memory is deallocated using free.

Q7. Explain the difference between 'call by value' and 'call by reference'

A.

Ans → Call by value	Call by address
(i) In this case passing value of variable. Ex - $x(a);$	(i) In this case passing address of variable. Ex - $x(&a);$
(ii) Can't change value of actual argument using formal argument.	(ii) Can change value of actual argument using formal argument.
(iii) No pointer are used.	(iii) pointer are used.
(iv) It requires more memory.	(iv) It requires less memory.
(v) It is less efficient.	(v) It is more efficient.

Q8. Write a function using pointers to exchange the values stored in two locations in the memory. I.e swapping call by reference

A.

```
#include <stdio.h>
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
int main() {
    int x = 5, y = 10;
    printf("Before swapping: x = %d, y = %d\n", x, y);
    // Call the swap function with addresses of x and y
    swap(&x, &y);
    printf("After swapping: x = %d, y = %d\n", x, y);
    return 0;
}
```

```
Before swapping: x = 5, y = 10
After swapping: x = 10, y = 5
```

Q9. What is the difference between an array of pointers and pointer to the array?

A.

Parameters	Pointer to an Array	Array of Pointers
Uses and Purposes	A user creates a pointer for storing the address of any given array.	A user creates an array of pointers that basically acts as an array of multiple pointer variables.
Alternative Names	It is alternatively known as an array pointer.	These are alternatively known as pointer arrays.
Allocation	One can allocate these during the run time.	One can allocate these during the compile time.
Initialization at Definition	You cannot initialize a pointer to the definition.	You can easily initialize an array at the definition level.
Nature	It is dynamic in nature.	It is static in nature.
Resizing	One can easily resize the allocated memory of a pointer later at any given time.	Once we declare the size of an array, we cannot resize it any time we want according to our requirements.
Type of Storage	A typical pointer variable is capable of storing only a single variable within.	The size of any given array decides the total number of variables that it can store within.

Q10. Write a program to accept a string and print the address of its beginning location.

A.

```
#include <stdio.h>
int main() {
    char str[100];
    // Input a string
    printf("Enter a string: ");
    scanf("%s", str);
    // Print the address of the beginning of the string
    printf("Address of the beginning of the string: %p\n", (void *)str);
    return 0;
}
```

```
Enter a string: Ani Das
Address of the beginning of the string: 0x7ffec211ac80
```

We print the address of the beginning of the string using the `%p` format specifier, casting `str` to `void*` to ensure proper printing of the address.

Q11. Write a C program to pass a pointer as function argument.

A.

```
#include <stdio.h>
void displayValue(int *ptr) {
    printf("Value at the address %p: %d\n", (void *)ptr, *ptr);
}
int main() {
    int num = 42;
    // Declare a pointer and initialize it with the address of 'num'
    int *ptr = &num;
    // Pass the pointer to the function
    displayValue(ptr);
    return 0;
}
```

```
Value at the address 0x7ffcc467de2c: 42
```

Q12. Define static, dynamic memory allocation and write a C program that allocates memory for a list of N characters at run time, and then print all vowels from the input data. A.

Static Memory Allocation:

Static Memory Allocation refers to the allocation of memory during the compile time of a program. The size and type of memory are determined at the program's compilation, and the variables are assigned fixed memory addresses. Arrays are a common example of static memory allocation in C.

```
int array[5]; // Static memory allocation for an integer array of size 5
```

Dynamic Memory Allocation:

Dynamic Memory Allocation refers to the allocation of memory during the runtime of a program. The size and type of memory are determined at runtime, allowing for flexibility in memory management. Dynamic memory allocation in C is achieved using functions like malloc, calloc, realloc, and free.

```
int *dynamicArray = (int *)malloc(5 * sizeof(int)); // Dynamic memory allocation for an integer array of size 5
```

C Program: Allocate Memory for a List of Characters and Print Vowels

```

#include <stdlib.h>
int main() {
    int n;
    // Input the number of characters
    printf("Enter the number of characters: ");
    scanf("%d", &n);
    // Allocate memory for a list of characters
    char *charList = (char *)malloc(n * sizeof(char));
    if (charList == NULL) {
        printf("Memory allocation failed.\n");
        return 1;
    }
    // Input characters
    printf("Enter the characters:\n");
    for (int i = 0; i < n; i++) {
        printf("Enter character %d: ", i + 1);
        scanf(" %c", &charList[i]);
    }
    // Print vowels from the input data
    printf("Vowels in the input data:\n");
    for (int i = 0; i < n; i++) {
        char currentChar = charList[i];
        // Check if the current character is a vowel
        if (currentChar == 'a' || currentChar == 'e' || currentChar == 'i' || currentChar == 'o' ||
            currentChar == 'u' ||
            currentChar == 'A' || currentChar == 'E' || currentChar == 'I' || currentChar == 'O' ||
            currentChar == 'U') {
            printf("%c ", currentChar);
        }
    }
    // Deallocate the allocated memory
    free(charList);
    return 0;
}

```

In this program:

- The user inputs the number of characters (n).
- Dynamic memory is allocated for an array of characters (charList) using malloc.
- The program then inputs the characters from the user.
- It prints the vowels from the input data.
- Finally, it deallocates the dynamically allocated memory using free.

Strings : String Manipulation

using user defined and library functions

Q13. Demonstration of string manipulation along with example using library functions(string.h,ctype.h)?

A.

1. Concatenation (`strcat`):

- **Theory:** Concatenation is the process of combining two strings into one. In C, `strcat` is used to concatenate the content of the source string to the end of the destination string.
- **Functioning:** `strcat(destination, source)` appends the characters of the source string to the end of the destination string, and it terminates the resulting string with a null character (`'\0'`).

2. Copying (`strcpy`):

- **Theory:** Copying involves duplicating the content of one string into another. The `strcpy` function is used for this purpose in C.
- **Functioning:** `strcpy(destination, source)` copies the characters of the source string to the destination string. It continues copying until it encounters a null character (`'\0'`) in the source string.

3. String Length (`strlen`):

- **Theory:** The length of a string is the number of characters in it, excluding the null character (`'\0'`). The `strlen` function calculates this length.
- **Functioning:** `strlen(string)` returns the length of the provided string by counting the characters until it reaches the null character (`'\0'`).

4. String Comparison (`strcmp`):

- **Theory:** String comparison involves determining the lexicographic order of two strings. The `strcmp` function is used for this purpose.

- **Functioning:** `strcmp(str1, str2)` compares two strings. It returns 0 if the strings are equal, a negative value if `str1` is lexicographically less than `str2`, and a positive value if `str1` is lexicographically greater than `str2`.

5. Substring Search (`strstr`):

- **Theory:** Substring search involves finding the first occurrence of a substring within a string. The `strstr` function performs this task.
- **Functioning:** `strstr(string, substring)` searches for the first occurrence of `substring` within `string` and returns a pointer to the start of the substring. If the substring is not found, it returns `NULL`.

6. String Tokenization (`strtok`):

- **Theory:** String tokenization involves breaking a string into a sequence of tokens based on a delimiter. The `strtok` function facilitates this process.
- **Functioning:** `strtok(string, delimiter)` returns a pointer to the first token found in `string` using `delimiter`. Subsequent calls with `NULL` as the first argument continue tokenizing the string.

7. Uppercase Conversion (`toupper`):

- **Theory:** Uppercase conversion involves converting characters in a string to uppercase. The `toupper` function performs this transformation.
- **Functioning:** `toupper(character)` returns the uppercase equivalent of the provided character. It works only for characters that have a lowercase counterpart.

8. Lowercase Conversion(`tolower`):

- **Theory:** Lowercase conversion involves converting characters in a string to lowercase. The `tolower` function performs this transformation.
- **Functioning:** `tolower(character)` returns the uppercase equivalent of the provided character. It works only for characters that have a lowercase counterpart.

9. Whitespace Stripping (`sscanf`):

- **Theory:** Whitespace stripping involves removing leading and trailing whitespace from a string. The `sscanf` function can be used for this purpose.
- **Functioning:** `sscanf(string, format, ...)` reads formatted data from the string. In the example, `%s` is used to read a sequence of non-whitespace characters, effectively stripping leading and trailing whitespace.

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
int main() {
```

```

// Example 1: Concatenation
char str1[20] = "DJ";
char str2[] = " Tillu";
strcat(str1, str2);
printf("1. Concatenation: %s\n", str1);

// Example 2: Copying
char source[] = "Mark ass";
char destination[20];
strcpy(destination, source);
printf("2. Copying: Copied String: %s\n", destination);

// Example 3: String Length
char text[] = "Lilly";
int length = strlen(text);
printf("3. String Length: %d\n", length);

// Example 4: String Comparison
char str3[] = "iron";
char str4[] = "rod";
int result = strcmp(str3, str4);
printf("4. String Comparison: %s\n", (result == 0) ? "Equal" : "Not Equal");

// Example 5: Substring Search
char sentence[] = "Oops Exams";
char *found = strstr(sentence, "Exam");
printf("5. Substring Search: %s\n", (found != NULL) ? "Substring found" : "Substring not found");

// Example 6: String Tokenization
char sentenceToTokenize[] = "Badass!!";
char *token = strtok(sentenceToTokenize, " ");
printf("6. String Tokenization: Tokens: ");
while (token != NULL) {
    printf("%s ", token);
    token = strtok(NULL, " ");
}
printf("\n");

// Example 7: Uppercase Conversion
char original[] = "Doraemon";
for (int i = 0; i < strlen(original); i++) {
    original[i] = toupper(original[i]);
}

```

```

}
printf("7. Uppercase Conversion: %s\n", original);
// Example 8: Lowercase Conversion
char orig[] = "ShinCHAN";
for (int i = 0; i < strlen(orig); i++) {
    orig[i] = tolower(orig[i]);
}
printf("8. Lowercase Conversion: %s\n", orig);
// Example 9: Whitespace Stripping
char textWithWhitespace[] = " Ibomma ";
char trimmedText[20];
sscanf(textWithWhitespace, "%s", trimmedText);
printf("9. Whitespace Stripping: %s\n", trimmedText);
return 0;
}

```

Q14. C program to check whether the String is palindrome or not? Even print reverse of a string without using pre defined function? (Write for this user defined function int string_palindrome(*char str) and without using pre defined find length, palindrome, reverse)

A.

```

#include <stdio.h>
// Function to check if a string is a palindrome
int isPalindrome(char str[]) {
    int left = 0;
    int right = 0;
    // Find the length of the string without using user defined function
    while (str[right] != '\0') {
        right++;
    }
    right--;
    // Check if the string is a palindrome
    while (left < right) {
        if (str[left] != str[right]) {
            return 0; // Not a palindrome
        }
        left++;
    }
}

```

```
right--;
}
return 1; // Palindrome
}
// Function to reverse a string
void reverseString(char str[]) {
int length = 0;
// Find the length of the string without using user defined function
while (str[length] != '\0') {
length++;
}
length--;
int start = 0;
int end = length;
// Swap characters to reverse the string
while (start < end) {
char temp = str[start];
str[start] = str[end];
str[end] = temp;
start++;
end--;
}
}
int main() {
char input[100];
// Input from the user
printf("Enter a string: ");
scanf("%s", input);
// Check if the string is a palindrome
if (isPalindrome(input)) {
printf("The string is a palindrome.\n");
} else {
printf("The string is not a palindrome.\n");
}
// Reverse the string
reverseString(input);
// Print the reversed string
printf("Reversed String: %s\n", input);
return 0;
}
```

```
Enter a string: GTA6
The string is not a palindrome.
Reversed String: 6ATG
```

```
Enter a string: ASSA
The string is a palindrome.
Reversed String: ASSA
```

Q15. Write a C program that counts the total number of Vowels.

A.

```
#include <stdio.h>
#include <ctype.h>
int countVowels(const char *str) {
    int count = 0;
    while (*str) {
        char current = tolower(*str);
        if (current == 'a' || current == 'e' || current == 'i' || current == 'o' || current == 'u')
        {
            count++;
        }
        str++;
    }
    return count;
}
int main() {
    char input[100];
    // Input from the user
    printf("Enter a string: ");
    scanf("%s", input);
    // Count vowels
    int vowelCount = countVowels(input);
    // Output
    printf("Total number of vowels: %d\n", vowelCount);
    return 0;
}
```

```
Enter a string: ascii
Total number of vowels: 3
```

Q16. Write a C program to compare two strings without using string functions.

A.

```
#include <stdio.h>
int compareStrings(const char *str1, const char *str2) {
    while (*str1 && *str2) {
        if (*str1 != *str2) {
            return 0; // Not equal
        }
        str1++;
        str2++;
    }
    return (*str1 == '\0' && *str2 == '\0'); // Both strings are equal if both have reached the end
}
int main() {
    char str1[100], str2[100];
    // Input from the user
    printf("Enter the first string: ");
    scanf("%s", str1);
    printf("Enter the second string: ");
    scanf("%s", str2);
    // Compare strings
    if (compareStrings(str1, str2)) {
        printf("The strings are equal.\n");
    } else {
        printf("The strings are not equal.\n");
    }
    return 0;
}
```

```
Enter the first string: Samrath
Enter the second string: Samrath
The strings are equal.
```

```
Enter the first string: Merchant
Enter the second string: Merchantine
The strings are not equal.
```

UNIT - V

Searching : Linear Search, Binary Search (Iterative & Recursive)

Q1. What is Linear Search, give its Time complexity, Tracing and Write a C program to perform linear search using Iterative and Recursive?

A. Linear Search:

- The **linear search** is an searching algorithm that iterates through the array **sequentially**, checking each element until it finds the target or reaches the end of the array.
- The **time complexity** of linear search is **$O(n)$** where 'n' is the number of elements in the array.

Linear Search Tracing :

For example: Consider the array `arr[] = {10, 50, 30, 70, 80, 20, 90, 40}` and `key = 30`

Step 1: Start from the first element (index 0) and compare **key** with each element (`arr[i]`).

- Comparing `key` with first element `arr[0]`. Since not equal, the iterator moves to the next element as a potential match.



Linear Search Algorithm



Compare key with `arr[0]`

- Comparing `key` with next element `arr[1]`. Since not equal, the iterator moves to the next element as a potential match.

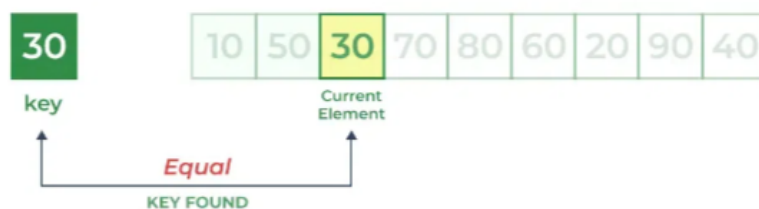


Linear Search Algorithm



Compare key with `arr[1]`

Step 2: Now when comparing `arr[2]` with `key`, the value matches. So the Linear Search Algorithm will yield a successful message and return the index of the element when `key` is found (here 2).



Linear Search Algorithm



Compare key with `arr[2]`

C program Code for Linear Iterative and Recursive :

```
#include <stdio.h>

// Linear Search Function (Iterative)
int linearSearchIterative(int arr[], int n, int target) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == target) {
            return i; // Return the index if the target is found
        }
    }
    return -1; // Return -1 if the target is not found
}

// Linear Search Function (Recursive)
int linearSearchRecursive(int arr[], int start, int end, int target) {
    if (start > end) {
        return -1; // Base case: target not found
    }

    if (arr[start] == target) {
        return start; // Return the index if the target is found
    }

    return linearSearchRecursive(arr, start + 1, end, target); // Recursive call for the rest
    of the array
}

int main() {
    int n;

    printf("Enter the number of elements in the array: ");
    scanf("%d", &n);

    int arr[n], target;

    printf("Enter the elements of the array:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Enter the element to search: ");
    scanf("%d", &target);
```

```

int iterativeResult = linearSearchIterative(arr, n, target);
if (iterativeResult != -1) {
    printf("Linear Search (Iterative): Element %d found at index %d\n", target,
iterativeResult);
} else {
    printf("Linear Search (Iterative): Element %d not found in the array\n", target);
}

// Linear Search Function Call (Recursive)
int recursiveResult = linearSearchRecursive(arr, 0, n - 1, target);
if (recursiveResult != -1) {
    printf("Linear Search (Recursive): Element %d found at index %d\n", target,
recursiveResult);
} else {
    printf("Linear Search (Recursive): Element %d not found in the array\n", target);
}

return 0;
}

```

O/P:

```

Enter the number of elements in the array: 4
Enter the elements of the array:
8      34      1      7
Enter the element to search: 1
Linear Search (Iterative): Element 1 found at index 2
Linear Search (Recursive): Element 1 found at index 2

Enter the number of elements in the array: 4
Enter the elements of the array:
8      34      1      7
Enter the element to search: 9
Linear Search (Iterative): Element 9 not found in the array
Linear Search (Recursive): Element 9 not found in the array

```

Linear Search (Iterative):

- The **linearSearchIterative** function iterates through the array **sequentially using a loop**, checking each element until it finds the target or reaches the end of the array.

Linear Search (Recursive):

- The **linearSearchRecursive** function uses recursion to perform linear search. It checks the **base case (start > end)** and returns -1 if the target is not found.
- If the current element is the target, it returns the index. Otherwise, it makes a recursive call for the rest of the array.

Q2. What is Binary Search, give its Time complexity, Tracing and Write a C program to perform binary search using Iterative and Recursive?

A. Binary Search:

- The **binary search** is an searching algorithm that assumes the **array is sorted before** and uses a **divide-and-conquer** approach to find the target element.
- It repeatedly divides the search range in half until the target is found or the range is empty.
- The **time complexity** of binary search is **$O(\log n)$** , making it more efficient than linear search for sorted arrays.

Binary Search Tracing :

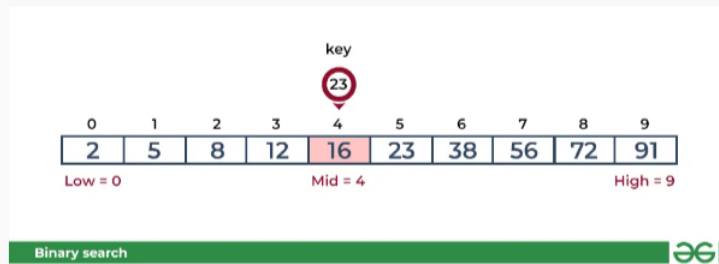
How does Binary Search work?

To understand the working of binary search, consider the following illustration:

Consider an array `arr[] = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91}`, and the **target = 23**.

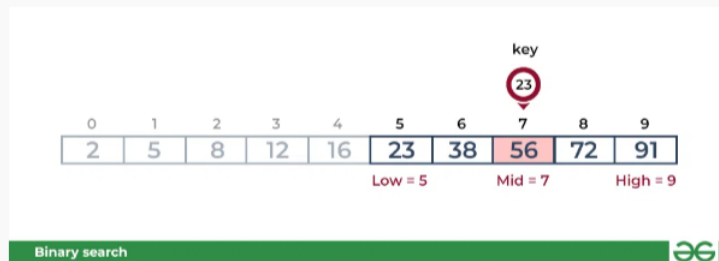
First Step: Calculate the mid and compare the mid element with the key. If the key is less than mid element, move to left and if it is greater than the mid then move search space to the right.

- Key (i.e., 23) is greater than current mid element (i.e., 16). The search space moves to the right.



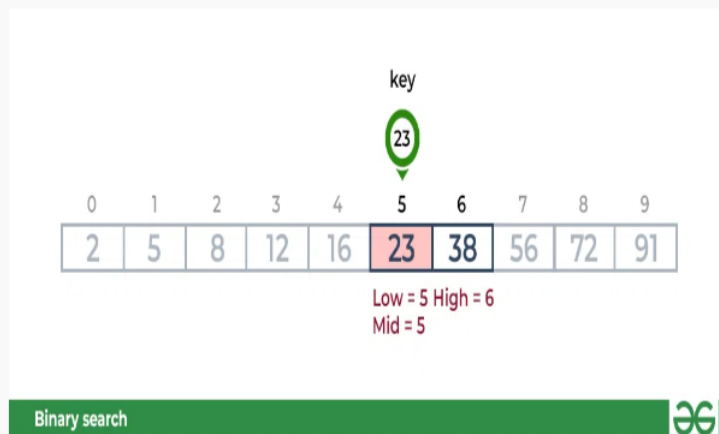
Binary Search Algorithm : Compare key with 16

- Key is less than the current mid 56. The search space moves to the left.



Binary Search Algorithm : Compare key with 56

Second Step: If the key matches the value of the mid element, the element is found and stop search.



Binary Search Algorithm : Key matches with mid

C program Code for Binary Iterative and Recursive :

```
#include <stdio.h>
// Binary Search Function (Iterative)
int binarySearchIterative(int arr[], int n, int target) {
    int left = 0;
    int right = n - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] == target) {
            return mid; // Return the index if the target is found
        }
        if (arr[mid] < target) {
            left = mid + 1; // Discard the left half
        } else {
            right = mid - 1; // Discard the right half
        }
    }
    return -1; // Return -1 if the target is not found
}

// Binary Search Function (Recursive)
int binarySearchRecursive(int arr[], int left, int right, int target) {
    if (left <= right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] == target) {
            return mid; // Return the index if the target is found
        }
        if (arr[mid] < target) {
            return binarySearchRecursive(arr, mid + 1, right, target); // Search in the right half
        } else {
            return binarySearchRecursive(arr, left, mid - 1, target); // Search in the left half
        }
    }
    return -1; // Return -1 if the target is not found
}

int main() {
    int n;
    printf("Enter the number of elements in the sorted array: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter the sorted elements of the array:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    int target;
    printf("Enter the element to search: ");
    scanf("%d", &target);
```

```
// Binary Search (Iterative)
int iterativeResult = binarySearchIterative(arr, n, target);
if (iterativeResult != -1) {
    printf("Binary Search (Iterative): Element %d found at index %d\n", target,
iterativeResult);
} else {
    printf("Binary Search (Iterative): Element %d not found in the array\n", target);
}
// Binary Search (Recursive)
int recursiveResult = binarySearchRecursive(arr, 0, n - 1, target);
if (recursiveResult != -1) {
    printf("Binary Search (Recursive): Element %d found at index %d\n", target,
recursiveResult);
} else {
    printf("Binary Search (Recursive): Element %d not found in the array\n", target);
}
return 0;
}
```

```
Enter the number of elements in the sorted array: 6
Enter the sorted elements of the array:
7      9      13      17      33      25
Enter the element to search: 33
Binary Search (Iterative): Element 33 found at index 4
Binary Search (Recursive): Element 33 found at index 4
```

```
Enter the number of elements in the sorted array: 6
Enter the sorted elements of the array:
7      9      13      17      33      25
Enter the element to search: 10
Binary Search (Iterative): Element 10 not found in the array
Binary Search (Recursive): Element 10 not found in the array
```

Binary Search (Iterative):

- The **binarySearchIterative** function iteratively performs binary search by maintaining the left and right pointers, updating them based on the comparison with the target.

Binary Search (Recursive):

- The **binarySearchRecursive** function uses recursion to perform binary search. It checks the base case ($\text{left} \leq \text{right}$) and returns -1 if the target is not found.
- If the current element is the target, it returns the index. Otherwise, it makes a recursive call in either the left or right half of the array.

Q3. Differences between Linear Search and Binary Search?
(Even you can categorise advantages and disadvantages if asked from this if asked)

LINEAR SEARCH
VERSUS
BINARY SEARCH

LINEAR SEARCH	BINARY SEARCH
An algorithm to find an element in a list by sequentially checking the elements of the list until finding the matching element	An algorithm that finds the position of a target value within a sorted array
Also called sequential search	Also called half-interval search and logarithmic search
Time complexity is $O(N)$	Time complexity is $O(\log_2 N)$
Best case is to find the element in the first position	Best case is to find the element in the middle position
It is not required to sort the array before searching the element	It is necessary to sort the array before searching the element
Less efficient	More efficient
Less complex	More complex

Sorting : Bubble Sort, Selection Sort, Insertion Sort, Quick Sort and Merge Sort

Q4. What is Bubble Sort, give its Time complexity, Tracing and Write a C program to perform bubble sort using Iterative and Recursive?

A.

Bubble Sort: Bubble Sort is a simple sorting algorithm that repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. The pass through the list is repeated until the list is sorted.

Time Complexity: The time complexity of Bubble Sort is $O(n^2)$ in the **worst and average cases**, and $O(n)$ in the **best case** (when the list is already sorted).

Bubble Sort Tracing : [Click here for video demonstration](#)

C program Code for Bubble Iterative and Recursive :

```
#include <stdio.h>
// Bubble Sort Function (Iterative)
void bubbleSortIterative(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                // Swap arr[j] and arr[j + 1]
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
// Bubble Sort Function (Recursive)
void bubbleSortRecursive(int arr[], int n) {
    // Base case: If only one element is remaining, return
    if (n == 1) {
        return;
    }
    // One pass of bubble sort, largest element will be at the end
    for (int i = 0; i < n - 1; i++) {
        if (arr[i] > arr[i + 1]) {
            // Swap arr[i] and arr[i + 1]
            int temp = arr[i];
            arr[i] = arr[i + 1];
            arr[i + 1] = temp;
        }
    }
    // Recursively call for the remaining elements
    bubbleSortRecursive(arr, n - 1);
}
// Function to print an array
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}
int main() {
    int arr[] = {5, 3, 1, 4, 2};
    int n = sizeof(arr) / sizeof(arr[0]);
    // Bubble Sort (Iterative)
    bubbleSortIterative(arr, n);
    printf("Bubble Sort (Iterative): ");
```

```

printArray(arr, n);
// Reset array for recursive version
int arrRecursive[] = {5, 3, 1, 4, 2};
// Bubble Sort (Recursive)
bubbleSortRecursive(arrRecursive, n);
printf("Bubble Sort (Recursive): ");
printArray(arrRecursive, n);
return 0;
}

```

O/P:

```

Bubble Sort (Iterative): 1 2 3 4 5
Bubble Sort (Recursive): 1 2 3 4 5

```

Q5. What is Selection Sort, give its Time complexity, Tracing and Write a C program to perform Selection sort?

A.

Selection Sort: Selection Sort is a simple sorting algorithm that divides the input array into two parts: a sorted and an unsorted region. The algorithm repeatedly selects the smallest (or largest, depending on the ordering) element from the unsorted region and swaps it with the first unsorted element. This process continues until the entire array is sorted.

Time Complexity: The time complexity of Selection Sort is **$O(n^2)$ in all cases**, as it involves nested loops.

Selection Sort Tracing : [Click here for video demonstration](#)

C program Code for Selection Sort:

```
#include <stdio.h>
// Selection Sort Function (Iterative)
void selectionSortIterative(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        // Swap arr[i] and arr[minIndex]
        int temp = arr[i];
        arr[i] = arr[minIndex];
        arr[minIndex] = temp;
    }
}
// Function to print an array
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}
int main() {
    int arr[] = {5, 3, 1, 4, 2};
    int n = sizeof(arr) / sizeof(arr[0]);
    // Selection Sort (Iterative)
    selectionSortIterative(arr, n);
    // Print the sorted array
    printf("Selection Sort (Iterative): ");
    printArray(arr, n);
    return 0;
}
```

```
Selection Sort (Iterative): 1 2 3 4 5
```

Q6. What is Insertion Sort, give its Time complexity, Tracing and Write a C program to perform Insertion sort?

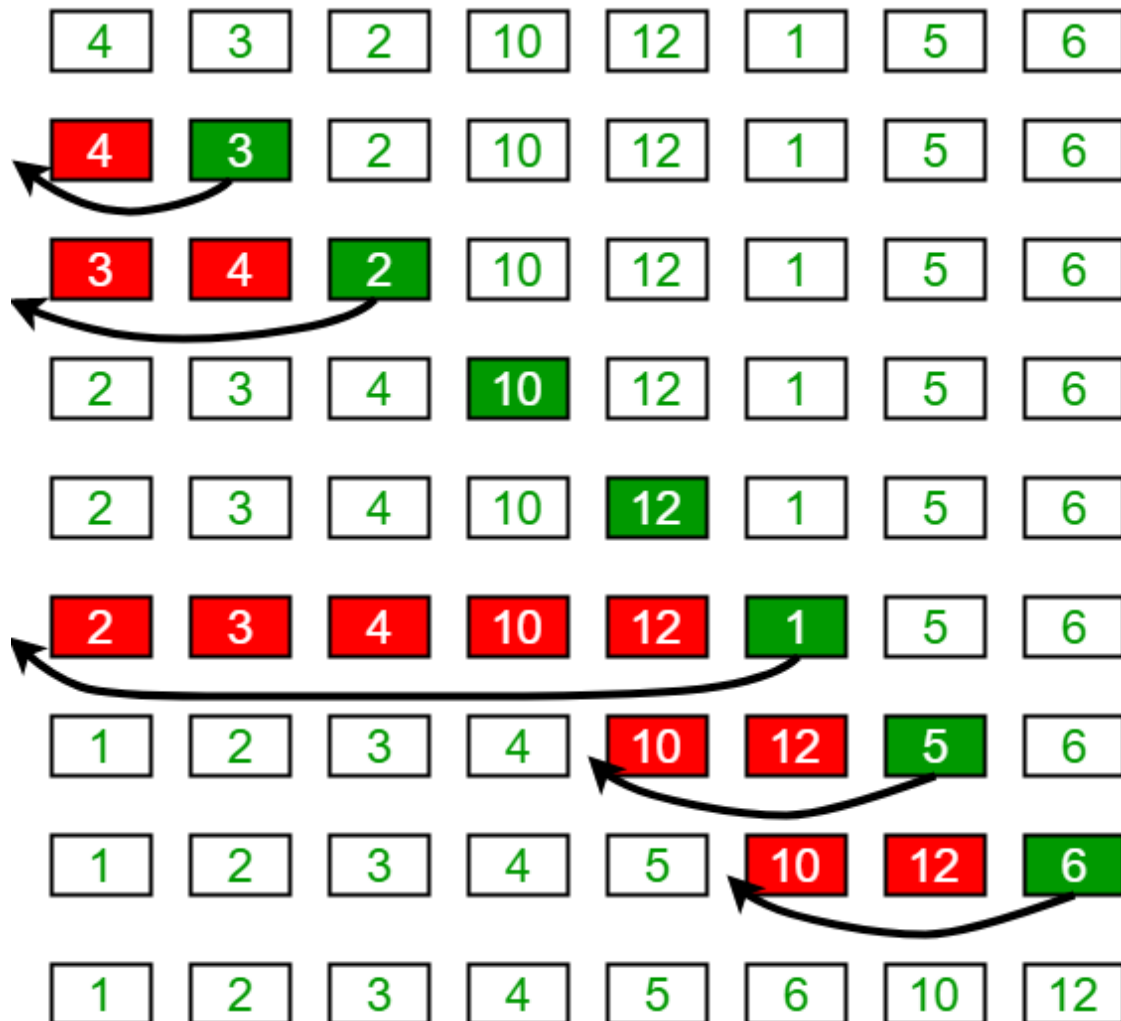
A.

Insertion Sort: Insertion Sort is a simple sorting algorithm that builds the final sorted array one item at a time. It is much less efficient on large lists than more advanced algorithms such as quicksort, heapsort, or merge sort. However, it has several advantages, including its simplicity, low overhead, and ability to efficiently sort partially sorted lists.

Time Complexity: The time complexity of Insertion Sort is $O(n^2)$ in the **worst and average cases**, and $O(n)$ in the **best case** (when the list is already sorted). It is an adaptive algorithm, meaning that it performs better when given a partially sorted list.

Insertion Sort Tracing : [Click here for video demonstration](#)

Insertion Sort Execution Example



C program Code for Insertion Sort:

```
#include <stdio.h>
// Insertion Sort Function
void insertionSort(int arr[], int n) {
    int i, key, j;
    for (i = 1; i < n; i++) {
        key = arr[i];
        j = i - 1;
        // Move elements of arr[0..i-1] that are greater than key to one position ahead of their
        // current position
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}
```

```

}
arr[j + 1] = key;
}
}
int main() {
int arr[] = {5, 3, 1, 4, 2};
int n = sizeof(arr) / sizeof(arr[0]);
// Insertion Sort
insertionSort(arr, n);
// Print the sorted array
printf("Insertion Sort: ");
for (int i = 0; i < n; i++) {
printf("%d ", arr[i]);
}
return 0;
}

```

```
Insertion Sort: 1 2 3 4 5
```

Q7. What is Quick Sort, give its Time complexity, Tracing and Write a C program to perform Quick sort?

A.

Quick Sort:

QuickSort is a highly efficient and widely used sorting algorithm that uses a **divide-and-conquer strategy**. It works by selecting a 'pivot' element from the array and partitioning the other elements into two sub-arrays, according to whether they are less than or greater than the pivot. The sub-arrays are then sorted recursively.

Time Complexity:

The **average time complexity** of QuickSort is **$O(n \log n)$** , making it faster than many other sorting algorithms. However, in the **worst case**, it can degrade to **$O(n^2)$** , but this is relatively rare and can be avoided with good pivot selection strategies and randomization.

Quick Sort Tracing : [Click here for video demonstration](#)

C program Code for Quick Sort:

```
#include <stdio.h>
// Function to swap two elements in an array
void swap(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
// Function to find the partition position (last element as the pivot)
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1; // Index of smaller element
    for (int j = low; j <= high - 1; j++) {
        // If current element is smaller than or equal to the pivot
        if (arr[j] <= pivot) {
            i++;
            // Swap arr[i] and arr[j]
            swap(&arr[i], &arr[j]);
        }
    }
    // Swap arr[i+1] and arr[high] (put the pivot in its correct position)
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}
// Function to implement QuickSort
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        // Find the partition position
        int pi = partition(arr, low, high);
        // Recursively sort the sub-arrays
        quickSort(arr, low, pi - 1);
    }
}
```

```

quickSort(arr, pi + 1, high);
}
}
int main() {
int arr[] = {5, 3, 1, 4, 2};
int n = sizeof(arr) / sizeof(arr[0]);
// QuickSort
quickSort(arr, 0, n - 1);
// Print the sorted array
printf("QuickSort: ");
for (int i = 0; i < n; i++) {
printf("%d ", arr[i]);
}
return 0;
}

```

```
QuickSort: 1 2 3 4 5
```

Q8. What is Merge Sort, give its Time complexity, Tracing and Write a C program to perform Merge sort?

A.

Merge Sort: Merge Sort is a popular **divide-and-conquer** sorting algorithm that divides the input array into two halves, recursively sorts each half, and then merges the sorted halves to produce a sorted output. It is a stable and efficient algorithm with a time complexity of $O(n \log n)$ in all cases (worst, average, and best).

Time Complexity: The time complexity of Merge Sort is **$O(n \log n)$** in **all cases**. This makes it more efficient than some

other sorting algorithms, especially for large datasets. It performs well on linked lists as well as arrays.

Merge Sort Tracing : [Click here for video demonstration](#)

C program Code for Merge Sort:

```
// Merge sort in C
#include <stdio.h>
// Merge two subarrays L and M into arr
void merge(int arr[], int p, int q, int r) {
// Create L ← A[p..q] and M ← A[q+1..r]
int n1 = q - p + 1;
int n2 = r - q;
int L[n1], M[n2];
for (int i = 0; i < n1; i++)
L[i] = arr[p + i];
for (int j = 0; j < n2; j++)
M[j] = arr[q + 1 + j];
// Maintain current index of sub-arrays and main array
int i, j, k;
i = 0;
j = 0;
k = p;
// Until we reach either end of either L or M, pick larger among
// elements L and M and place them in the correct position at A[p..r]
while (i < n1 && j < n2) {
if (L[i] <= M[j]) {
arr[k] = L[i];
i++;
} else {
arr[k] = M[j];
j++;
}
k++;
}
// When we run out of elements in either L or M,
// pick up the remaining elements and put in A[p..r]
while (i < n1) {
arr[k] = L[i];
i++;
k++;
}
while (j < n2) {
arr[k] = M[j];
j++;
}
```

```

k++;
}
}
// Divide the array into two subarrays, sort them and merge them
void mergeSort(int arr[], int l, int r) {
    if (l < r) {
        // m is the point where the array is divided into two subarrays
        int m = l + (r - l) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        // Merge the sorted subarrays
        merge(arr, l, m, r);
    }
}
// Driver program
int main() {
    int arr[] = {6, 5, 12, 10, 9, 1};
    int size = sizeof(arr) / sizeof(arr[0]);
    mergeSort(arr, 0, size - 1);
    printf("Merge Sort: ");
    for (int i = 0; i < size; i++)
        printf("%d ", arr[i]);
}

```

```
Merge Sort: 1 5 6 9 10 12
```

Made with ❤ by Telegram [@oxrogerthat](https://t.me/oxrogerthat)